

TASK 1

1. Install MySQL Workbench and Take Screenshot:

Steps:

1. Download and install **MySQL Community Server** and **MySQL Workbench**.
2. Open **MySQL Workbench**.
3. Connect to your **Local Instance MySQL**.
4. In the **Schemas** panel, you should see default databases such as:
 - information_schema
 - mysql
 - performance_schema
 - sys
5. **Take a screenshot** showing these databases and submit it.

2. Create Database and Users Table:

-- Create Database

```
CREATE DATABASE InstaClone;
```

-- Select Database

```
USE InstaClone;
```

-- Create Users Table

```
CREATE TABLE Users (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(50),
```

```
email VARCHAR(100),  
followers_count INT  
);
```

3. Insert Sample Data:

```
INSERT INTO Users  
(user_id, username, email, followers_count)  
VALUES  
(1, 'travel_with_riya', 'riya@gmail.com', 15420),  
(2, 'foodie_rahul', 'rahul@gmail.com', 9825),  
(3, 'techguru_amit', 'amit@gmail.com', 32100);
```

4. Create Posts Table with Foreign Key:

```
CREATE TABLE Posts (  
    post_id INT PRIMARY KEY,  
    user_id INT,  
    caption VARCHAR(255),  
    post_date DATE,  
    FOREIGN KEY (user_id)  
    REFERENCES Users(user_id)  
);
```

5. Comparison of MySQL, PostgreSQL, Oracle, and SQLite:

MySQL

MySQL is one of the most popular open-source relational database systems. It is widely used for websites, blogs, and web applications because of its speed and ease of use. It is

commonly used with PHP and the LAMP stack. Examples include WordPress, e-commerce websites, and social media applications.

PostgreSQL

PostgreSQL is an advanced open-source relational database known for its reliability and powerful features. It supports complex queries, large databases, and advanced data types. It is commonly used in financial systems, GIS applications, analytics, and enterprise web applications.

Oracle Database

Oracle Database is a commercial relational database management system designed for large enterprises. It offers high security, scalability, and excellent performance for mission-critical applications. It is widely used in banking, healthcare, government organizations, and multinational companies.

SQLite

SQLite is a lightweight, serverless relational database stored in a single file. It requires no installation or configuration, making it ideal for small applications. It is commonly used in mobile applications (Android and iOS), embedded systems, desktop software, and browsers.

TASK 2

1.Create the restaurants Table:

```
CREATE TABLE restaurants (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    location VARCHAR(100),  
    rating DECIMAL(2,1)  
);
```

2. Add the average_cost Column:

```
ALTER TABLE restaurants
```

```
ADD average_cost INT;
```

3. Drop the restaurants Table:

```
DROP TABLE restaurants;
```

4. Create the playlists Table:

```
CREATE TABLE playlists (  
    playlist_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(50),  
    created_by VARCHAR(50),  
    created_at DATE  
);
```

TASK 3

1. Create the Restaurants Table and Insert 5 Sample Records:

```
-- Create the Restaurants table
```

```
CREATE TABLE Restaurants (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100),  
    cuisine VARCHAR(50),  
    rating DECIMAL(2,1),
```

```
city VARCHAR(50)
);

-- Insert sample records

INSERT INTO Restaurants (name, cuisine, rating, city)
VALUES
('Spice Villa', 'Indian', 4.5, 'Ahmedabad'),
('Pizza Hub', 'Italian', 4.2, 'Ahmedabad'),
('Dragon Bowl', 'Chinese', 3.8, 'Surat'),
('Taco Fiesta', 'Mexican', 4.6, 'Vadodara'),
('Sushi House', 'Japanese', 3.4, 'Ahmedabad');
```

2. Update the Rating of Spice Villa to 4.7:

```
UPDATE Restaurants
SET rating = 4.7
WHERE name = 'Spice Villa';
```

3. Delete Restaurants with Rating Less Than 3.5:

```
DELETE FROM Restaurants
WHERE rating < 3.5;
```

4. Select the Top 2 Restaurants in Ahmedabad with Rating Above 4.0:

```
SELECT *
FROM Restaurants
```

WHERE city = 'Ahmedabad'

AND rating > 4.0

LIMIT 2;

TASK 4

1. Create the Playlist Table and Insert 5 Sample Records:

-- Create Playlist table

CREATE TABLE Playlist (

id INT AUTO_INCREMENT PRIMARY KEY,

song_name VARCHAR(100),

artist VARCHAR(100),

genre VARCHAR(50),

play_count INT

);

-- Insert sample records

INSERT INTO Playlist (song_name, artist, genre, play_count)

VALUES

('Blinding Lights', 'The Weeknd', 'Pop', 250),

('Shape of You', 'Ed Sheeran', 'Pop', 180),

('Lose Yourself', 'Eminem', 'Hip-Hop', 220),

('Believer', 'Imagine Dragons', 'Rock', 140),

('Starboy', 'The Weeknd', 'Hip-Hop', 120);

2. Display Only song_name and artist (Alias as Singer):

```
SELECT  
  
    song_name,  
  
    artist AS Singer  
  
FROM Playlist;
```

3. Find All Pop Songs with play_count Greater Than 100 and Sort by play_count (Descending):

```
SELECT *  
  
FROM Playlist  
  
WHERE genre = 'Pop'  
  
    AND play_count > 100  
  
ORDER BY play_count DESC;
```

4. Count the Number of Hip-Hop Songs:

```
SELECT COUNT(*) AS Total_HipHop_Songs  
  
FROM Playlist  
  
WHERE genre = 'Hip-Hop';
```

5. Show the Total Number of Plays for Each Genre:

```
SELECT  
  
    genre,  
  
    SUM(play_count) AS Total_Plays  
  
FROM Playlist
```

GROUP BY genre;

TASK 5

1. Create the Tables and Insert Sample Data:

-- Create influencers table

CREATE TABLE influencers (

influencer_id INT PRIMARY KEY,

name VARCHAR(100)

);

-- Create posts table

CREATE TABLE posts (

post_id INT PRIMARY KEY,

influencer_id INT,

caption VARCHAR(255),

FOREIGN KEY (influencer_id) REFERENCES influencers(influencer_id)

);

-- Insert influencers

INSERT INTO influencers (influencer_id, name)

VALUES

(1, 'Riya Sharma'),

(2, 'Aman Patel'),

(3, 'Neha Verma');

```
-- Insert posts (2 posts for each influencer)

INSERT INTO posts (post_id, influencer_id, caption)

VALUES

(101, 1, 'Enjoying the beach! '),

(102, 1, 'Morning workout '),

(103, 2, 'Delicious homemade pizza '),

(104, 2, 'Travel diaries '),

(105, 3, 'Tech gadgets review '),

(106, 3, 'Weekend vibes ');
```

2. INNER JOIN – Show Each Post with Influencer Name:

```
SELECT

    influencers.name,

    posts.caption

FROM influencers

INNER JOIN posts

ON influencers.influencer_id = posts.influencer_id;
```

3. LEFT JOIN – Show All Influencers and Their Posts:

```
SELECT

    influencers.name,

    IFNULL(posts.caption, 'No Posts') AS caption

FROM influencers

LEFT JOIN posts
```

ON influencers.influencer_id = posts.influencer_id;

4. RIGHT JOIN – Show All Posts with Corresponding Influencer Name:

```
SELECT
    IFNULL(influencers.name, 'Unknown Influencer') AS name,
    posts.caption
FROM influencers
RIGHT JOIN posts
ON influencers.influencer_id = posts.influencer_id;
```

TASK 6

1. GROUP BY – Total Number of Orders Placed by Each User:

```
SELECT
    user_id,
    COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

2. GROUP BY and HAVING – Restaurants with Average Rating Above 4.0:

```
SELECT
```

```
name,  
AVG(rating) AS average_rating  
FROM restaurants  
GROUP BY name  
HAVING AVG(rating) > 4.0;
```

3. Subquery – Find Users Who Made a Single Payment Above ₹2000:

```
SELECT DISTINCT user_id  
FROM payments  
WHERE user_id IN (  
    SELECT user_id  
    FROM payments  
    WHERE amount > 2000  
);
```

4. Nested Subquery – Movies with Higher Average Rating Than the Overall Average:

```
SELECT  
name,  
AVG(rating) AS average_rating  
FROM movies  
GROUP BY name  
HAVING AVG(rating) > (  
    SELECT AVG(rating)
```

```
FROM movies  
);
```

TASK 7

1. Create the Restaurant Table:

```
CREATE TABLE Restaurant (  
    id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    location VARCHAR(100) NOT NULL,  
    cuisine VARCHAR(50)  
);
```

2. Create the FoodOrder Table with a Foreign Key:

```
CREATE TABLE FoodOrder (  
    order_id INT PRIMARY KEY,  
    restaurant_id INT,  
    user_id INT,  
    order_total DECIMAL(10,2),  
    FOREIGN KEY (restaurant_id) REFERENCES Restaurant(id)  
);
```

3. Create the SpotifyUser Table and Insert 6 Rows:

```
-- Create SpotifyUser table
```

```
CREATE TABLE SpotifyUser (  
    user_id INT PRIMARY KEY,  
    username VARCHAR(50) UNIQUE,  
    email VARCHAR(100) NOT NULL,  
    subscription_type VARCHAR(20)  
);  
  
-- Insert sample data  
INSERT INTO SpotifyUser (user_id, username, email, subscription_type)  
VALUES  
(1, 'musiclover01', 'musiclover01@gmail.com', 'Premium'),  
(2, 'rockfan22', 'rockfan22@gmail.com', 'Free'),  
(3, 'popqueen', 'popqueen@gmail.com', 'Premium'),  
(4, 'hiphopking', 'hiphopking@gmail.com', 'Premium'),  
(5, 'jazzbeats', 'jazzbeats@gmail.com', 'Free'),  
(6, 'melodymaker', 'melodymaker@gmail.com', 'Premium');
```

4. Create a View TopSpendersView:

```
CREATE VIEW TopSpendersView AS  
SELECT  
    user_id AS username,  
    order_total  
FROM FoodOrder  
WHERE order_total > 1000;
```

TASK 8

1. Declare Variables and Print User Name and Age:

```
SET SERVEROUTPUT ON;

DECLARE

    v_name VARCHAR2(50);

    v_age NUMBER;

BEGIN

    v_name := 'Divy Patel';

    v_age := 21;


    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);

    DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);

END;

/
```

2. Use IF Statement to Check Order Amount:

```
SET SERVEROUTPUT ON;

DECLARE

    v_order_amount NUMBER := 650;

BEGIN

    IF v_order_amount > 500 THEN

        DBMS_OUTPUT.PUT_LINE('Eligible for free delivery');
```

```
ELSE
    DBMS_OUTPUT.PUT_LINE('Delivery charges apply');
END IF;
END;
/
```

3. Use a LOOP to Print Numbers 1 to 5:

```
SET SERVEROUTPUT ON;

DECLARE
    counter NUMBER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(counter);
        counter := counter + 1;

        EXIT WHEN counter > 5;
    END LOOP;
END;
/
```

4. Declare a Cursor to Display Product Names:

```
SET SERVEROUTPUT ON;

DECLARE
    CURSOR product_cursor IS
```

```
SELECT product_name
FROM PRODUCTS;

v_product_name PRODUCTS.product_name%TYPE;
BEGIN

OPEN product_cursor;

LOOP

    FETCH product_cursor INTO v_product_name;

    EXIT WHEN product_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE(v_product_name);

END LOOP;

CLOSE product_cursor;

END;

/
```

5. Handle Division by Zero Using Exception Handling:

```
SET SERVEROUTPUT ON;

DECLARE

    num1 NUMBER := 100;

    num2 NUMBER := 0;

    result NUMBER;

BEGIN
```

```
result := num1 / num2;

DBMS_OUTPUT.PUT_LINE('Result: ' || result);

EXCEPTION

    WHEN ZERO_DIVIDE THEN

        DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');

END;

/
```

TASK 9

1. Insert a New Order and Commit the Transaction:

```
-- Insert a new order

INSERT INTO Orders (order_id, user_id, order_total)

VALUES (101, 1, 850.00);

-- Save the transaction

COMMIT;

-- Verify the order

SELECT * FROM Orders

WHERE order_id = 101;

:
```

2. Zomato-Style Order Process Using ROLLBACK:

-- Insert two order items

```
INSERT INTO OrderItems (item_id, order_id, item_name, quantity)
```

```
VALUES (1, 101, 'Veg Pizza', 1);
```

```
INSERT INTO OrderItems (item_id, order_id, item_name, quantity)
```

```
VALUES (2, 101, 'Cold Drink', 2);
```

-- Undo both inserts

```
ROLLBACK;
```

-- Verify that no new items exist

```
SELECT * FROM OrderItems
```

```
WHERE order_id = 101;
```

3. Flipkart-Style Shopping Cart Using SAVEPOINT:

-- Add first product

```
INSERT INTO Cart (cart_id, product_name, quantity)
```

```
VALUES (1, 'Laptop', 1);
```

-- Create a savepoint

```
SAVEPOINT sp1;
```

-- Add second product

```
INSERT INTO Cart (cart_id, product_name, quantity)
```

```
VALUES (2, 'Mouse', 1);
```

```
-- Undo only the second insert
```

```
ROLLBACK TO sp1;
```

```
-- Save remaining changes
```

```
COMMIT;
```

```
-- Display final cart contents
```

```
SELECT * FROM Cart;
```

4. Trigger to Automatically Deduct Wallet Balance:

```
-- Wallet table
```

```
CREATE TABLE Wallet (
```

```
    user_id INT PRIMARY KEY,
```

```
    balance DECIMAL(10,2)
```

```
);
```

```
-- Transactions table
```

```
CREATE TABLE WalletTransactions (
```

```
    transaction_id INT PRIMARY KEY,
```

```
    user_id INT,
```

```
    purchase_amount DECIMAL(10,2)
```

```
);
```

```
-- Trigger
```

```
CREATE TRIGGER trg_wallet_deduction
AFTER INSERT ON WalletTransactions
FOR EACH ROW
UPDATE Wallet
SET balance = balance - NEW.purchase_amount
WHERE user_id = NEW.user_id;
```

5. Buggy Trigger, Test Scenario, and Corrected Trigger:

Buggy Trigger

```
CREATE TRIGGER trg_buggy_wallet
AFTER INSERT ON WalletTransactions
FOR EACH ROW
UPDATE Wallet
SET balance = NEW.purchase_amount
WHERE user_id = NEW.user_id;
```

Test Case

```
-- Initial wallet balance
INSERT INTO Wallet VALUES (1, 5000);

-- Purchase of ₹1000
INSERT INTO WalletTransactions VALUES (1, 1, 1000);

-- Check balance
SELECT * FROM Wallet;
```

Corrected Trigger

```
CREATE TRIGGER trg_wallet_deduction
AFTER INSERT ON WalletTransactions
FOR EACH ROW
UPDATE Wallet
SET balance = balance - NEW.purchase_amount
WHERE user_id = NEW.user_id;
```

Mini Project

1. Create the playlists Table:

```
CREATE TABLE playlists (
    playlist_id INT PRIMARY KEY,
    user_id INT,
    name VARCHAR(100),
    created_at DATETIME,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

2. Insert Three Sample Records:

```
INSERT INTO playlists (playlist_id, user_id, name, created_at)
VALUES
(1, 101, 'Workout Mix', '2026-08-01 08:30:00'),
(2, 101, 'Chill Vibes', '2026-08-05 19:00:00'),
(3, 102, 'Top Hits', '2026-08-10 10:15:00');
```

3. Update the Playlist Name:

```
UPDATE playlists  
SET name = 'Evening Chill'  
WHERE user_id = 101  
AND name = 'Chill Vibes';
```

4. Delete the Workout Mix Playlist:

```
DELETE FROM playlists  
WHERE user_id = 101  
AND name = 'Workout Mix';
```

5. Create the GetMonthlyPlaylistCount Stored Procedure:

```
DELIMITER //  
  
CREATE PROCEDURE GetMonthlyPlaylistCount(  
    IN p_user_id INT,  
    IN p_month INT  
)  
  
BEGIN  
  
    SELECT COUNT(*) AS total_playlists  
    FROM playlists  
    WHERE user_id = p_user_id  
        AND MONTH(created_at) = p_month;  
  
END //  
  
DELIMITER ;
```

